Mathematical Limits of Transactional Unlinkability and Research Gaps in Public Ledger Privacy

1 The Standard UTXO Model vs. Collaborative Transaction Schemas

The mathematical limits of transactional unlinkability are deeply rooted in the architecture of Bitcoin's Unspent Transaction Output (UTXO) model [1]. In this framework, a standard transaction is formally modeled as an ordered triple $t = (A, B, c)$ [2]. A is a multiset of transaction inputs, where each input $(a_n, A_n) \in A$ is an ordered pair consisting of the address A_n and the value $a_n \geq 0$ [3]. B is a multiset of transaction outputs, where each output $(b_m, B_m) \in B$ represents an ordered pair of the receiving address B_m and the value $b_m \geq 0$ [3]. c is the transaction fee, representing the difference between total inputs and total outputs, defined as $c = \sum a_n - \sum b_m \geq 0$ [3].

In a standard transaction, the widely-used multi-input heuristic assumes that all input addresses $A_n \in A$ are controlled by a single entity who possesses the respective private keys [4, 5]. Collaborative transaction schemas (such as CoinJoin or Shared Send Mixers) deliberately disrupt this model to enhance anonymity [6, 7]. They merge the UTXOs of multiple distinct users into a single shared transaction where there are multiple inputs ($|A| > 1$) and multiple outputs ($|B| > 1$) [8]. By aggregating funds, these schemas break the fundamental assumption of the multi-input heuristic, tangling the money flows so that the direct correspondence between the original senders and receivers is no longer apparent [7, 9].

1.1 Evaluating Post-Mix Spending with the One-Sided Chamfer Distance

Even when collaborative transactions successfully obscure initial linkages, users eventually spend their mixed funds in post-mix transactions, known as CoinJoin Spending Transactions (CSTs) [10]. Users frequently construct CSTs by combining mixed outputs that originate from multiple distinct CoinJoin processes spanning different time frames [10].

To link CSTs back to a single user, analysts model the timestamps of the CoinJoin inputs utilized in the CST as 1-dimensional point clouds in Euclidean space [11]. To evaluate the similarity between a targeted CST (with timestamp set U) and another CST in the blockchain (with timestamp set V), the one-sided Chamfer distance metric is applied [12]:

$$d_C(U, V) = \frac{1}{\|U\|} \sum_{u_i \in U} \min_{v_j \in V} \|u_i - v_j\| \tag{1}$$

This specific metric is highly effective for identifying linked transactional behavior because it mathematically accommodates point clouds of unequal lengths [12, 13]. The one-sided nature of the formula ensures that outlier timestamps in set V (which have a large Euclidean distance from points in U) do not skew the similarity score, provided that points with a low Euclidean distance are also present [12, 13].

1.2 Arbitrary Value Mixing as a Subset-Sum Partition Problem

A central vulnerability of collaborative transactions is that they can often be mathematically untangled if users mix non-standardized, arbitrary values [7]. The untangling process is modeled as searching for a connectable pair of subsets $A' \subset A$ and $B' \subset B$ [14]. For a pair to be connectable, the intended expenses of the input subset must mathematically account for their actual outputs and a fraction of the fee [14]. Formally, the sum of the selected inputs must be greater than or equal to the sum of the selected outputs:

$$\sum_{(a_i, A_i) \in A'} a_i - \sum_{(b_j, B_j) \in B'} b_j \geq 0 \tag{2}$$

(And specifically, the difference cannot exceed the transaction fee c) [14].

This creates a variant of the subset-sum partition problem, wherein the untangling algorithm attempts to decompose the overall transaction graph into minimal acceptable partitions (subtransactions that cannot be further subdivided) [15, 16]. Sub-mapping resolution occurs when analysts find a unique minimal partition that breaks the larger transaction into isolated, valid sub-components [17]. If such a unique partition exists (meaning $|P| > 1$), the transaction is classified as separable [17]. When a transaction is separable, the obfuscation is entirely defeated, and the flows are resolved back to their original senders and receivers [17, 18].

While identifying ambiguous shared send transactions is an NP-complete problem, transactions that mix arbitrary values rarely generate sufficient mathematical ambiguity to protect users [19]. Because of this vulnerability, modern anonymity protocols enforce standardized identical output denominations, artificially generating vast numbers of valid sub-mappings to render the transaction mathematically ambiguous and ensuring the subset-sum problem cannot be resolved to a single true partition [20, 21].

2 Identified Research Gaps in Public Ledger Privacy

2.1 Gap 1: Taproot-Upgrade Clustering Bypasses and False Positives

Context: The recent Bitcoin Taproot soft fork introduced Pay-to-Taproot (P2TR) scripts and Schnorr signatures, which allow the aggregation of keys and signatures [22]. This privacy enhancement renders single-user public keys and multi-user aggregated public keys indistinguishable on the blockchain, effectively bypassing traditional multiple-input clustering heuristics [22]. While researchers have proposed block number-based address clustering (relying on the 6-confirmation rule) to associate trust relationships, this method remains highly prone to false positives due to the pseudo-anonymity of the system [23, 24].

Novel Academic Contribution: To address this gap, research could focus on a hybrid methodology combining the recovery of Taptree structures via spent Tapscripts and the tracking of reused public keys. Integrating this with external concepts like hyperbolic graph embeddings could provide a novel way to model the hidden structural probabilities of address ownership in continuous hierarchical spaces.

Validation Methodology: Validation should benchmark the new model against baseline heuristics, such as the standard common spending heuristic and naive block-number clustering [25, 26]. The framework must be evaluated using empirical ground-truth datasets, such as real-world criminal investigation records from the Korean National Police Agency or GraphSense tag packs [27, 28]. Evaluation must prioritize the F1-score, the address reduction ratio, and the pure contribution of the new heuristic [29-31].

2.2 Gap 2: Multi-Dimensional Post-Mix Spending (CST) Deanonymization

Context: Users frequently obscure their payment trails by combining unspent transaction outputs (UTXOs) from different CoinJoin processes into subsequent post-mix spending transactions, known as CoinJoin Spending Transactions (CSTs) [10]. Currently, linking CSTs to a single controlling entity relies on a one-sided Chamfer distance metric that evaluates only the timestamps of the spent mixed outputs [11, 12]. This temporal-only focus leaves a significant vulnerability, as it ignores other structural data that could resolve ambiguous transactions.

Novel Academic Contribution: A high-impact contribution would be the development of Graph Neural Networks (GNNs) that integrate time-based attributes with multi-dimensional transaction features [32]. This model would structurally analyze the connected CoinJoin graph, expanding the parameters to include used amounts, input counts, and denomination frequencies alongside timeframes [33, 34].

Validation Methodology: The study should construct datasets of CSTs spanning multiple protocols, specifically isolating addresses that receive funds from at least two CSTs via Dash, Whirlpool, and Wasabi 2.0 implementations [35]. The GNN model should be measured against the baseline timestamp-only one-sided Chamfer distance metric. Researchers must quantify the superior linkability of the GNN by calculating the F1-score and reporting robust false-positive reduction rates [29].

2.3 Gap 3: Machine Learning Class Imbalances in Illicit Activity Detection

Context: The application of supervised and unsupervised machine learning to the Bitcoin user graph frequently struggles with severe data limitations and class imbalances [36, 37]. Because fraudulent or mixed transactions make up a tiny fraction of total volume, models frequently fail to learn discriminative features, leading them to misclassify profoundly illegal businesses as legitimate activities [36].

Novel Academic Contribution: Research could introduce a self-supervised Deep Graph Infomax (DGI) framework combined with highly discriminative outlier features to circumvent the lack of balanced labelled data [32]. Specifically, the model would incorporate advanced engineered features like the Shortest-Path-to-CoinJoin, degree per active day, and average UTXO age, which have proven to be significant indicators of malicious behaviour [33, 34].

Validation Methodology: To handle the class imbalance, resampling techniques such as Adaptive Synthetic (ADASYN) oversampling should be applied during pre-processing [37]. The algorithm should be rigorously tested against baseline classifiers like Random Forest (RF), K-Nearest Neighbour (KNN), and standard K-means [37, 38]. Performance must be strictly quantified using Precision-Recall parameters and F1-scores [29].

3 Python Code Implementations

3.1 Vectorized One-Sided Chamfer Distance

This metric evaluates post-mix spending similarity by comparing the timestamps of CoinJoin inputs. It measures the mean minimum distance from points in a target transaction (U) to the points in another transaction (V) [12].

```python
import numpy as np
from scipy.spatial.distance import cdist

def calculate_chamfer_distance(U: np.ndarray, V: np.ndarray) -> float:
    """
    Computes the one-sided Chamfer distance similarity between two multisets
    of parent transaction timestamps U and V.

    Math:
    d_C(U, V) = (1 / ||U||) * sum_{u_i in U} min_{v_j in V} ||u_i - v_j||
    """
    # Reshape arrays to 2D column vectors (N, 1) as required by scipy
    U_col = np.asarray(U).reshape(-1, 1)
    V_col = np.asarray(V).reshape(-1, 1)

    # Vectorized computation of the Euclidean distance matrix
    dist_matrix = cdist(U_col, V_col, metric='euclidean')

    # Find the minimum distance for each point in U to any point in V
    min_distances = np.min(dist_matrix, axis=1)

    # Calculate the average of these minimum distances
    chamfer_dist = np.mean(min_distances)

    return float(chamfer_dist)
```

3.2 Subset-Sum Partitioning for Arbitrary Value Transactions

To mathematically untangle collaborative transactions, analysts must identify connectable pairs of input subsets (A') and output subsets (B') where $0 \leq \sum A' - \sum B' \leq c$ [14].

```python
def resolve_submappings(inputs: list[float], outputs: list[float], fee: float)
    -> list[tuple]:
    """
    Recursively solves the subset-sum balance partition problem to find
        connectable
```

```
    submappings in a shared send transaction.
    """
    valid_partitions = []

    def backtrack(in_idx: int, out_idx: int, current_A: list[float], current_B:
        list[float]):
        sum_A = sum(current_A)
        sum_B = sum(current_B)

        # Base check: If current non-empty subsets satisfy the connectability
            condition
        if len(current_A) > 0 and len(current_B) > 0:
            diff = sum_A - sum_B
            if 0 <= diff <= fee:
                valid_partitions.append((current_A[:], current_B[:]))

        # Recursive branch 1: Expand the input subset (A')
        for i in range(in_idx, len(inputs)):
            current_A.append(inputs[i])
            backtrack(i + 1, out_idx, current_A, current_B)
            current_A.pop()

        # Recursive branch 2: Expand the output subset (B')
        if in_idx == len(inputs):
            for j in range(out_idx, len(outputs)):
                current_B.append(outputs[j])
                backtrack(in_idx, j + 1, current_A, current_B)
                current_B.pop()

    # Initialize the recursive backtracking search
    backtrack(0, 0, [], [])

    return valid_partitions
```

3.3 Machine Learning Classifier Pipeline

Implementation of a class-weighted Random Forest classifier to handle class scarcity without resorting exclusively to synthetic resampling, robustly measuring precision via F1-scores and PR-AUC [29, 37].

```
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, GridSearchCV,
    StratifiedKFold
from sklearn.metrics import average_precision_score, f1_score,
    classification_report

def train_ledger_classifier(X: np.ndarray, y: np.ndarray):
    """
    Trains a class-weighted Random Forest classifier to detect anomalous
    collaborative transactions under severe class imbalance.
    """
    # 1. Stratified Split to preserve the severely imbalanced class
        distribution
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )

    # 2. Define the Random Forest Classifier
    # Using class_weight='balanced' automatically adjusts weights inversely
    # proportional to class frequencies.
    rf_clf = RandomForestClassifier(class_weight='balanced', random_state=42)

    # 3. Define Hyperparameter Grid for Automated Tuning
```

```
    param_grid = {
        'n_estimators': [39-41],
        'max_depth': [None, 10, 20, 30],
        'min_samples_split': [42-44],
        'min_samples_leaf': [42, 45, 46]
    }

    cv_strategy = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

    # 4. Automated Hyperparameter Tuning using GridSearchCV
    grid_search = GridSearchCV(
        estimator=rf_clf,
        param_grid=param_grid,
        scoring='f1_macro',
        cv=cv_strategy,
        n_jobs=-1,
        verbose=1
    )

    grid_search.fit(X_train, y_train)
    best_model = grid_search.best_estimator_

    # 5. Evaluate the model
    y_pred = best_model.predict(X_test)
    y_pred_proba = best_model.predict_proba(X_test)[:, 1]

    # 6. Compute PR-AUC and F1-Score
    pr_auc = average_precision_score(y_test, y_pred_proba)
    f1 = f1_score(y_test, y_pred)

    metrics = {
        'PR-AUC': pr_auc,
        'F1-Score': f1
    }

    return best_model, metrics
```